Paper Critique: LoRA: Low-Rank Adaptation of Large Language Models

Full Name: Mathis DOUTRE
Student ID: 904-04-8015
Paper #: 5
Title : LoRA: Low-Rank Adaptation of Large Language Models

---

1 Problems Adressed

The paper LoRA, by Hu et al. (2021)[1] tackles the practical challenges of fine-tuning very large pre-trained language models for specific problems. As models became larger (such as GPT-3 with 175 billion parameters), **full fine-tuning** (updating all model weights for each new task) became impractical due to computational and storage costs. Fine-tuning a 175B-parameter model for every task means deploying a separate model instance of that size for each use case, which is increasingly expensive in terms of memory and storage. As an example, a single GPT-3 checkpoint in FP16 is 350GB, maintaining 100 fully fine-tuned variants would require 35 TB. This creates a bottleneck: Storing hundreds of such large models is simply infeasible.

Another problem the paper tackles is the fine-tuning **efficiency barrier**. Updating billions of weights not only consumes enormous GPU memory (gradients and optimizer states), but also demands time-consuming training. Fine-tuning GPT-3 "requires 780 GB of GPU memory" (65B model, 16-bit), illustrating the hardware barrier to adapting such models. Even if one has the resources to fine-tune a single large model, doing so for many tasks multiplies the cost.

Researchers have proposed parameter-efficient adaptation methods to address these issues, but those methods came with trade-offs. One class of methods is inserting small adapter layers into the network (Houlsby et al., 2019 [2]), so that only the adapter parameters (actually representing a small fraction of the model) are trained for each task. However, these techniques "often introduce inference latency"([1]) by adding extra layers in the forward pass. Another approach is prefix-tuning or prompt-tuning, which keeps the model weights fixed and instead learns task-specific embeddings or "prompts" (Li and Liang, 2021 [3]). This usually reduce trainable parameters but may "reduce the model's usable sequence length"([1]) (since prompt tokens consume input space) and can be less effective than fine-tuning. Indeed, prior efficient tuning methods often failed to reach the full fine-tuning accuracy, indicating in practice a quality vs. efficiency trade-off. Bias-only tuning (Zaken et al., 2021 [4]) or

fine-tuning only the last layers reduces the overhead cost but often underperforms full tuning on complex tasks.

The core problem the paper addresses is: **How can we adapt large language models to new tasks with fewer trainable parameters and lower compute costs, while maintaining performance?** The authors specifically target the scenario of massive Transformer-based LLMs where duplicating the entire model for each task is not practical. They seek a solution that lowers memory and computation overhead for fine-tuning, without sacrificing model quality or increasing inference time. The paper addresses the bottleneck of deploying LLMs at scale.

2 New Idea and Strengths

LoRA (Low-Rank Adaptation) introduces an elegant solution to those problems : instead of adjusting all of a model's parameters when training for a specific task, it freezes the original weights and learns a **small additive update** for each weight matrix, constrained to be **low-rank**. This idea is motivated by earlier findings that the changes needed to fine-tune a large model might lie in a **low-dimensional subspace** (Aghajanyan et al., 2020 [5]). In other words, the authors propose that "the change in weights during model adaptation has a low intrinsic rank"[1], even if the weight matrices themselves are high-rank and high-dimensional.

LoRA implements this by representing the weight update ΔW as a product of two much smaller matrices: B and A (of dimension $d \times r$ and $r \times d$ respectively, with $W$of size $d \times d$), as shown in Figure 1. Here r is a chosen rank much smaller than d. During fine-tuning, the original weight W_0 remains unchanged, and only the small matrices A and B are trained, yielding an updated weight $W_{\text{ft}} = W_0 + \Delta W = W_0 + B \times A$. This low-rank parameterization reduces drastically the number of trainable parameters: instead of $d \times d$ parameters to learn for ΔW, LoRA needs only $2dr$ parameters (which for small r is orders of magnitude smaller).

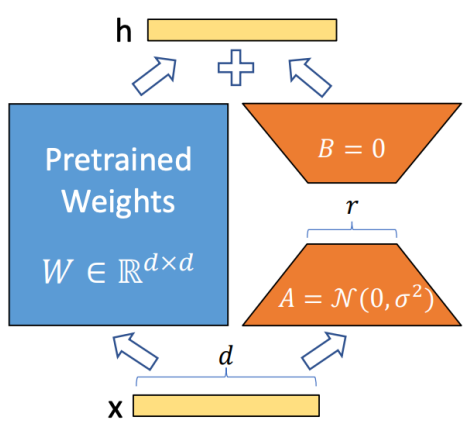

Figure 1: LoRA approach: W is the frozen pre-trained matrix; B and A are trainable low-rank factors with $\Delta W = BA$. Only A, B are updated during fine-tuning and merged into W for inference.

Strengths and innovations of LoRA:

By tuning low-rank matrices instead of the full matrix, LoRA **reduces the number of trainable parameters** by many orders of magnitude. For GPT-3, LoRA uses $17M$ trainable parameters, which is **10,000x fewer than the base model**. The resulting deltas are only tens of megabytes (e.g. 35 MB) as opposed to hundreds of gigabytes for a full model. This allow hundreds of task-specific models to be stored with minimal overhead. Freezing most weights also **reduces training memory** (e.g. in their GPT-3 setup, from $1.2TB$ to $350GB$).

LoRA **reduces trainable parameters and compute**: with most weights frozen, back-propagation and optimizer state are needed only for the small A and B matrices. Because 99.99% of parameters remain fixed, gradients and optimizer buffers are not computed and not stored for them. In GPT-3 experiments, this resulted to up to a 3× reduction in GPU memory usage compared with full fine-tuning, making large-model adaptation possible on more modest hardware and enabling more parallel runs. LoRA *"lowers the hardware barrier to entry"* for fine-tuning large models [1].

After training, **LoRA merges the low-rank update into the base weights**, $W_{ft} = W_0 + BA$, so inference uses a standard weight matrix with no added latency and behavior **equivalent** to full fine-tuning. In contrast, methods like prefix-tuning [3] or adapters add tokens or layers at inference, increasing compute and potentially slowing throughput.

LoRA often **matches or surpasses full fine-tuning** despite using far fewer trainable parameters. On GLUE with RoBERTa/DeBERTa and on GPT-2/GPT-3 across tasks such as WikiSQL, MultiNLI, and SAMSum, LoRA achieves comparable or better accuracy. As shown in Table 2, a GPT-3 LoRA model with only 4.7M trainable parameters reaches higher accuracy than full fine-tuning on WikiSQL (73.4 vs. 73.8), matches or exceeds performance on MultiNLI (91.7 vs. 89.5), and achieves the best ROUGE scores on SAMSum. LoRA also outperforms other efficient tuning methods (BitFit, prefix-based, adapters) while remaining orders of magnitude smaller. These results support the claim that **effective fine-tuning updates are low rank**.

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($Adapter^H$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($Adapter^H$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**
GPT-3 (LoRA)	37.7M	**74.0**	**91.6**	53.4/29.2/45.1

Figure 2: LoRA compared with full fine-tuning and other parameter-efficient methods on GPT-3 across WikiSQL, MultiNLI, and SAMSum (from Hu et al., 2021 [1]).

LoRA **scales well with model and task complexity**. In Figure 3, accuracy on WikiSQL and MNLI increases with rank (i.e., more trainable parameters) and exceeds other methods for the same parameter budget. LoRA achieve strong results at low ranks and continuing to improve as rank grows. Others methods often plateau or degrade when given more trainable parameters. This indicates LoRA **uses limited parameters efficiently**.

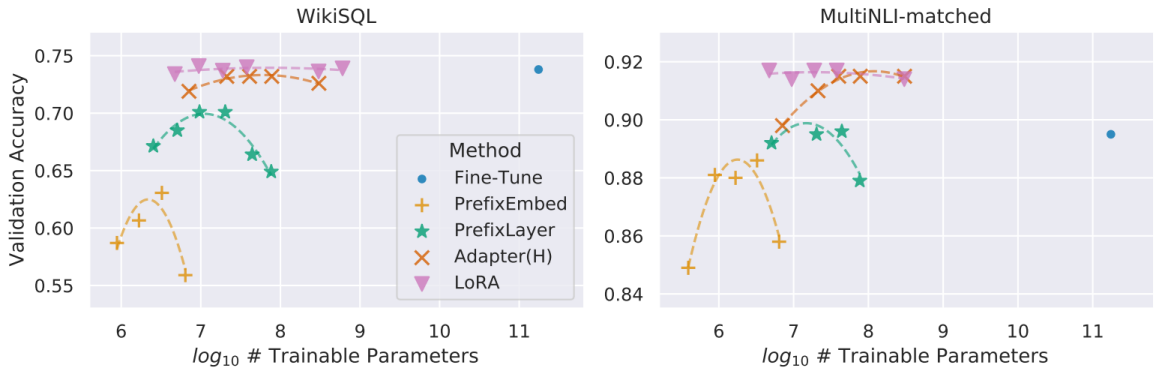

Figure 3: Validation accuracy vs. number of trainable parameters for GPT-3 (175B) on WikiSQL and MNLI-matched. LoRA attains higher accuracy at comparable budgets and scales smoothly with rank (from Hu et al., 2021 [1]).

With the base model frozen, task-specific LoRA modules (B, A pairs) can be swapped during inference with negligible overhead, enabling a library of fine-tuned models on a single checkpoint.
This **modularity** also means LoRA composes with other strategies: for example, using prefix-tuning prompts alongside LoRA-modified weights, or with Compacter's Kronecker-factored adapters [6] to further cut trainable parameters [1].

LoRA is **simple** to implement and requires no architectural changes, making it easy to reason about. An open-source PyTorch implementation further lowers the adoption barrier [7].

3 Weaknesses and Suggestions

Choosing the rank r is critical: very low ranks can underfit task-specific changes, while high ranks reduce LoRA's parameter savings. Although the paper states low ranks (1-8) are often enough and returns diminish for high values, some methods show non-monotonic behavior at larger budgets [8]. A possible future work is automatic rank selection to a parameter or memory budget, reducing manual tuning and adapting to the task complexity.

In the reported implementation, LoRA is **applied primarily to attention weights**, with MLP layers, LayerNorms, and biases **kept frozen** for simplicity and parameter efficiency. This restriction **may limit performance** on tasks that rely on adaptations in the frozen components. A natural extension is to evaluate low-rank adaptations beyond attention. For example selectively applying LoRA to MLP blocks, LayerNorm parameters, or biases, and to measure when such extensions yield meaningful gains compared to the added parameter cost.

LoRA is **optimized for single-task deployment**: when updates are merged ($W_{ft} =$

$W_0 + BA$), **only one adapter (A,B) can be active**, preventing a single forward pass from batching examples that require different adapters. The alternative is to **avoid merging** and select the adapter on the fly (meaning computing $W_0x + BAx$) which introduces some **inference overhead**.

Because the base weights remain frozen, **noise from pretraining can persist** and may be amplified in the adapted model (this is called **"catastrophic inheritance"**). Mitigations strategies include adding regularization to the LoRA updates to constrain deviation from the base model: BA-LoRA (Chang et al., 2024 [9]), aims to preserve knowledge while reducing noise.

LoRA's effectiveness **depends on hyperparameters and initialization**. Hu et al. initializes $B = 0$ and A randomly so $\Delta W = BA = 0$ at step 0, starting the tuning from the original weights. However, this can slow convergence in short fine-tuning runs (Huang and Balestriero (2024) [10]). Implementations also use a scaling factor α to control the magnitude of BA; poor choices can harm the optimization. Recent variants (ALLoRA [10]) replace manual scaling and dropout with adaptive, learning rates that normalize updates, improving efficiency.

Furthermore, **extending LoRA beyond NLP** seems promising because many domains face similar fine-tuning costs. Research reports applications to CNNs, speech models, and scientific pretrained models, suggesting the approach generalizes across modalities [11]. Overall, LoRA addresses a central fine-tuning cost with a simple low-rank update mechanism.

References

[1] EdwardJ. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Lu Wang, Sheanand Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. URL https://arxiv.org/abs/2106.09685.

[2] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. *arXiv preprint arXiv:1902.00751*, 2019. URL https://arxiv.org/abs/1902.00751.

[3] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021. URL https://arxiv.org/abs/2101.00190.

[4] Elad BenZaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. *arXiv preprint arXiv:2106.10199*, 2021. URL https://arXiv.org/abs/2106.10199.

[5] Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. *arXiv preprint arXiv:2012.13255*, 2020. URL https://arxiv.org/abs/2012.13255.

[6] RabeehKarimi Mahabadi, James Henderson, and Sebastian Ruder. Compacter: Efficient low-rank hypercomplex adapter layers. In *Advances in Neural Information Processing Systems (NeurIPS) 2021*, 2021. URL https://arXiv.org/abs/2106.04647.

[7] Microsoft. Lora: Code for low-rank adaptation of large language models (github repository). https://github.com/microsoft/LoRA, 2025.

[8] Rohan Sukumaran, Aarash Feizi, Adriana Romero-Soriano, and Golnoosh Farnadi. Fairlora: Unpacking bias mitigation in vision models with fairness-regularized low-rank adaptation. In *ICLR 2025 (OpenReview)*, 2025. URL https://openreview.net/forum?id=pB3KeBCnQs. Under review, OpenReview preprint.

[9] Yupeng Chang, Yi Chang, and Yuan Wu. Ba-lora: Bias-alleviating low-rank adaptation to mitigate catastrophic inheritance in large language models. *arXiv preprint arXiv:2408.04556*, 2024. URL https://arxiv.org/abs/2408.04556.

[10] Hai Huang and Randall Balestriero. Allora: Adaptive learning rate mitigates lora fatal flaws. *arXiv preprint arXiv:2410.09692*, 2024. URL https://arXiv.org/abs/2410.09692.

[11] Menglin Yang, Jialin Chen, Yifei Zhang, Jiahong Liu, Jiasheng Zhang, Qiyao Ma, Harshit Verma, Qianru Zhang, Min Zhou, Irwin King, and Rex Ying. Low-rank adaptation for foundation models: A comprehensive review. *arXiv preprint arXiv:2501.00365*, 2025. URL https://arXiv.org/pdf/2501.00365.